

AGI JOBS v0

The Operating System for AGI Work — verifiable execution, deterministic settlement, operator control.

AGIJobsv0 • AGI-Alpha-Agent-v0 • AGI-Alpha-Node-v0

α -WU: canonical unit of verified work

\$AGIALPHA: stake • settle • coordinate

Docs & code: github.com/MontrealAI/AGIJobsv0

META-AGENTIC Master
Presentation

Executive Summary

MASTER PRESENTATION

An always-on command lattice for verifiable AI work at network scale.

What it is

- Job OS: contracts + paymasters + orchestration services
- α -WU metering + peer validation → settlement-grade outputs
- Operator and owner control: pause, parameters, upgrades

Why it matters

- Turns AI output into auditable work with reproducible evidence
- Pricing and settlement become deterministic, not negotiated
- Shared productivity index for capacity planning and policy

What to do next

- Deploy the stack and register identity (ENS)
- Integrate via APIs/SDKs; consume proofs + receipts
- Extend: validators, dashboards, simulations, security tests

The Problem

MASTER PRESENTATION

AI work is difficult to verify, price, and settle across organizations.

What breaks today

- Compute $\neq$ verified work; outputs lack standardized evidence
- Hardware/model heterogeneity makes pricing inconsistent
- Audit trails are non-standard; reproducing runs is costly
- Settlement and disputes rely on trust, tickets, and vendors

What institutions require

- Deterministic execution + reproducible environments
- Tamper-evident telemetry and artifact integrity proofs
- Clear roles, accountability, and dispute pathways
- Governance levers: parameters, upgrades, emergency stop

The thesis

- Define a canonical unit of verified work (α -WU), attest it with peers, and settle it on-chain — so “AI labor” becomes measurable infrastructure.

The Vision

MASTER PRESENTATION

Make work programmable: measure → verify → settle → evolve.

PRIMITIVE

Verifiable Work

UNIT

α -WU

SETTLEMENT

\$AGIALPHA

What this enables

- A verifiable market for autonomous work: jobs → artifacts → attestations → settlement.
- A shared productivity index ($\Sigma \alpha\text{-WU} / \text{epoch}$) to measure throughput and reliability.
- Controlled autonomy: policies, circuit breakers, and governance-defined parameters.

Three Repositories, One System

MASTER PRESENTATION

Agent cognition → Work OS → Deployment. One coherent surface.

AGI-Alpha-Agent-v0

- Meta-agentic orchestration patterns
- Evaluation, planning, and iterative improvement loops
- Research-to-production interface for job execution

AGIJobsv0 (command center)

- Contracts kernel + paymasters + governance levers
- Orchestrator + agent gateway + dashboards (control plane)
- CI v2 assurance wall: main as deployable truth

AGI-Alpha-Node-v0

- Deployment runtime: execution, metering, artifact packaging
- Validator and sentinel services + observability wall
- Operator UX: deploy → register → participate

AGENT

JOBS OS

NODE

What is AGI Jobs v0?

MASTER PRESENTATION

A deployable work OS that binds agents, infrastructure, and settlement into one control plane.

Jobs OS = contracts + services

- Publishes jobs, escrows funds, and routes settlement by policy
- Meters work (α -WU) and stores signed usage + artifact proofs
- Coordinates validation, disputes, and reputation signals
- Exposes APIs, dashboards, simulations, and operator tooling

Owner & operator control

- Deterministic owner authority: pause, upgrades, parameters
- Air-gapped CLI workflows and multisig-ready bundles
- Policy gating, rate limits, and role separation by design
- CI-backed assurance: merges blocked if invariants break

Outputs

- Settlement-grade receipts: validated α -WU + signed artifacts
- Audit packs: logs, hashes, and provenance for each job
- Payments in \$AGIALPHA via paymaster + escrow release
- Reputation & discovery signals for routing and governance

Design Principles

MASTER PRESENTATION

Deployable truth, governed control, and auditable work.

Deterministic execution

- Reproducible environments + pinned dependencies
- Deterministic telemetry for every action
- Fail-fast checks: identity, stake, contract invariants

Verifiable identity

- ENS/DID anchored identities for nodes, agents, and roles
- Signed usage reports + tamper-evident audit trails
- Sybil resistance through staking and validator oversight

Owner-controlled economics

- \$AGIALPHA for staking, settlement, and coordination
- Parameterized fees, splits, and burn — governed and auditable
- Slashing aligns incentives with correctness

Institutional readiness

- Operator consoles + dashboards (read-only by default)
- Incident playbooks, key management, safe upgrades
- CI v2 guardrails: enforce gates before merge

Roles in the Market

MASTER PRESENTATION

Clear duties. Clear accountability. Composable participation.

Requester

- Posts jobs and funds escrow (in \$AGIALPHA)
- Defines SLOs and acceptance constraints
- Receives artifacts + validated receipts

Worker

- Executes jobs deterministically (node/agent runtime)
- Publishes artifacts (hash/IPFS) + signed usage
- Claims settlement after validation

Validator

- Attests via commit–reveal and quality scoring
- Confirms SLO + output validity; flags disputes
- Slashable for faults or dishonesty

End-to-End Job Lifecycle

MASTER PRESENTATION

From job request to validated artifacts and settlement receipts.



Result

- Each job yields artifacts, validated α -WU, and a settlement receipt — then settles deterministically via \$AGIALPHA.

α -Work Units (α -WU)

MASTER PRESENTATION

Canonical, hardware-agnostic measure of verified work — parameterized by policy.

Definition

- $\alpha\text{-WU} = \text{GPU}_s \times \text{gflops_norm} \times \text{ModelTier} \times \text{SLO_pass} \times \text{QV}$ (reference formulation)

Compute

- GPU_s : seconds of compute consumed
- gflops_norm : normalized capacity (A100 = 1.0)

Difficulty

- ModelTier : multiplier by model class
- Small = 0.8 • Base = 1.0 • Frontier = 1.3+ (illustrative)

Quality

- SLO_pass : latency/uptime adherence (0–1)
- QV : peer-validated output score (0–1)

Metering & Attestation

MASTER PRESENTATION

From runtime telemetry to settlement-grade receipts.

Metering sidecars

- Collects metering signals (compute, time, I/O, model tier)
- Builds usage report + artifact hash manifest
- Signs reports with node/agent identity keys (ENS/DID)
- Submits usage to the contracts kernel via gateway

Validator network

- Commit–reveal attestations with scored evidence
- Confirms SLO_pass and QV; opens dispute windows
- Slashes fraud or faulty attestations
- Finalizes α -WU for the epoch

Integrity

- Artifacts are content-addressed (hash/IPFS). Decisions are logged. Settlement follows validation — not promises.

\$AGIALPHA Utility: Stake • Settle • Coordinate

MASTER PRESENTATION

A work-credit for network operation — not equity, not profit rights.

Stake

- Bond participation
- Activate roles (worker/validator)
- Sybil resistance
- Slashable accountability

Settle

- Pay for jobs in \$AGIALPHA
- Escrow → release after validation
- Paymaster policy + fee routing + protocol burn
- Operator withdrawals

Coordinate

- Epoch accounting (α -WU)
- Incentive splits
- Reputation signals
- Governance parameters

Utility token only: required for network operation. No equity, no profit rights, no claims on an entity.

Token Coupling: \$AGIALPHA ↔ α -WU

MASTER PRESENTATION

Supply and incentives remain tethered to verifiable work.

Emission

- Epoch distribution proportional to validated α -WU
- Requires stake + role activation
- Aligns issuance with work

Settlement

- Jobs priced in α -WU; paid in \$AGIALPHA
- Escrow → release after validation
- Routes fees to workers/validators/treasury

Burn

- Small protocol fraction burned
- Links scarcity to usage
- Counterbalances emissions

Index + Metrics

- α -Productivity Index = $\Sigma \alpha\text{-WU} / \text{epoch}$
- Synthetic Labor Yield (SLY) = $\alpha\text{-WU validated} / \$\text{AGIALPHA circulating}$
- Public dashboards: emission, burn, validator scores

Productivity as a Public Metric

MASTER PRESENTATION

A network-wide index you can audit: $\Sigma \alpha$ -WU per epoch.

α -Productivity Index

- Definition: total validated α -WU per epoch
- Tracks throughput and reliability over time
- Enables capacity planning, governance tuning, and transparency

Dashboards (examples)

- α -WU / epoch (by tier, by region, by policy)
- Escrow + treasury flows, emission + burn
- Validator participation + QV distributions
- SLO compliance (latency/uptime) + incident history

Why it matters

- Transforms “AI work” into an auditable operational primitive.
- Lets institutions reason about productivity with the same discipline used for traditional infrastructure.

On-chain Architecture

MASTER PRESENTATION

Contracts provide identity, rules, settlement, and accountability.

Identity & Role Registry

- Register identities (ENS/DID) and activate roles
- Stake custody + slashing hooks
- Policy constraints and routing metadata

Jobs + Escrow + Metering

- Job posting, escrow, and settlement conditions
- Signed usage submission + artifact hash manifests
- Validator commit–reveal attestations + dispute windows

Index + Treasury + Incentives

- Productivity index: $\sum \alpha \cdot \text{WU} / \text{epoch}$
- Incentive manager: rewards, splits, and burns
- Fee routing: workers / validators / treasury

Off-chain Runtime

MASTER PRESENTATION

Services that execute, route, validate, and observe — end-to-end.

Orchestrator services

- Deterministic job orchestration (agents, tools, containers)
- Metering sidecars: usage capture + signature sealing
- Artifact packaging: hashes + immutable pointers
- Telemetry exporters for dashboards and audits

Agent gateway + consoles

- Agent gateway APIs (REST/WS) for jobs + receipts
- Operator consoles (`apps/`) for monitoring and control
- Indexers/subgraph for read-only analytics

Sentinels & validators

- Validator coordination (commit-reveal) + scoring
- Sentinel/thermostat services for health, drift, and policy
- Dispute evidence packaging for slashing and resolution

Safety & Operator Control

MASTER PRESENTATION

High autonomy, bounded by policy, keys, and circuit breakers.

Safety by design

- Policy gating, allowlists, and rate limits across execution
- Role-scoped permissions with least-privilege defaults
- Slashable accountability for misreporting or fraud
- Deterministic incident hooks: pause, quarantine, rollback

Operator supremacy

- Owner command center for pause, upgrades, and parameters
- Air-gapped workflows; artifacts exportable for governance review
- Multisig-ready transaction bundles and audit trails
- Operator override is final — autonomy is never sovereign

Audit & Compliance

MASTER PRESENTATION

Tamper-evident logs, reproducible builds, and forensics readiness.

Audit posture

- Signed receipts: usage, artifact hashes, and validator attestations
- Provenance lattice: CI-backed reports and release signers
- Reproducible builds + SBOMs + pinned toolchains
- Dispute evidence packs: slashing-ready, auditor-readable

Operational readiness

- Status wall: CI guardrails and branch protection parity
- Monitoring: SLO, drift, validator participation, paymaster health
- Key rotation, role separation, and access controls
- Versioned upgrades with rollback strategy

Governance & Transparency

MASTER PRESENTATION

Institutional posture: measurable, auditable, steerable.

Dashboards

- α -WU / epoch (by tier, region, policy)
- Escrow, settlement, treasury flows
- Validator participation + quality distributions
- CI assurance wall + release provenance

Governance controls

- Tune fees, splits, burn, and incentive schedules
- Adjust α -WU parameters (tiers, SLO, scoring)
- Upgrade modules via owner-controlled releases
- Emergency pause / circuit breaker

Audit & compliance

- Signed artifacts and tamper-evident decision logs
- Reproducible builds + SBOMs
- Multisig approvals archived with artifacts
- Forensics-ready evidence for disputes

Safety by design

- Slashing for misbehavior and malicious attestations
- Policy gating and rate limiting
- Key custody best practices
- Operator override is final

Integration Surface

MASTER PRESENTATION

Where developers, operators, and protocols plug in.

Identity

- ENS identities for nodes, agents, and roles
- DID keys for signing usage and artifact manifests
- Reputation and discovery registries

Artifacts

- Content-addressed outputs (hashes + immutable pointers)
- Telemetry attestations and receipts for audits
- Reproducible build proofs (SBOM + pinned toolchains)

APIs

- Agent gateway APIs (REST/WS) to contracts + receipts
- SDKs for job submission, routing, and validation hooks
- Metrics exporters (Prometheus/OpenTelemetry) + dashboards

Representative Use Cases

MASTER PRESENTATION

Verifiable, auditable AI labor for real operations.

Work types

- Settlement-grade inference services (SLO-bound)
- Data transformation + QA pipelines with attestations
- Continuous monitoring, anomaly triage, and remediation
- Simulation, evaluation, and procurement of model work

Why this stack

- Deterministic settlement and dispute minimization
- Evidence packs for compliance and oversight
- Modular roles and policy gating
- Owner-controlled upgrades with audit trails

Roadmap

MASTER PRESENTATION

From v0 kernel to scaled, governed production.

v0 (now)

- Contracts kernel + job lifecycle primitives
- Agent gateway, orchestrator, and reference dashboards
- Owner command center + CI assurance lattice

v1 (next)

- Validator decentralization + disputes / slashing evidence
- Paymaster policies, SDKs, and integration playbooks
- Observability, sentinels, and SLO tooling
- Security hardening + audits

v2 (scale)

- Tiered markets, routing, and reputation maturation
- Multi-chain / rollup deployment options
- Advanced simulations and stress tests
- Ecosystem growth: agents, nodes, and job markets

How to Participate

MASTER PRESENTATION

Run infrastructure, integrate jobs, or harden the assurance lattice.

Operators

- Deploy runtime services (node/orchestrator/gateway)
- Register ENS identity, stake, and activate roles
- Operate validators/sentinels; monitor dashboards
- Enforce policy and upgrade via owner command center

Builders

- Integrate: submit jobs via APIs/SDKs; consume receipts
- Build: dashboards, validators, simulations, reference apps
- Harden: reproducibility, SBOMs, pinned deps, tests
- Contribute: contracts, services, security, documentation

Repos: AGIJobsv0 • AGI-Alpha-Agent-v0 • AGI-Alpha-Node-v0

Make work verifiable. Make work programmable.

AGI Jobs v0: metered work, peer validation,
deterministic settlement — with owner-controlled governance.

github.com/MontrealAI/AGIJobsv0

agialpha.com

agialpha.com

Deterministic control, transaction safety, and audit-ready artifacts.

- Source of truth for pause/unpause and upgrades
- Parameter matrix: fees, treasury routes, validator policy
- Ownership invariants enforced by CI before merge
- Operator override preserved by design
- Deterministic CLIs for emitting calldata and bundles
- Dry-run previews and dependency-ordered module updates
- Multisig-ready plans (Safe) and air-gapped workflows
- Operator consoles consume the same artifacts
- Markdown/JSON authority matrices for auditors
- Signed change artifacts: what changed and why
- Reports exported under `reports/` for review
- Upgrades rehearsed and matched to CI assurance jobs

Owners, operators, and automation converge on one verifiable surface.

- Agent gateway APIs for job intake and receipts
- Operator consoles (`apps/`) for monitoring and control
- Owner CLI + scripts for deterministic actions
- Read-only analytics for observers and auditors

- Contracts kernel + modules (registries, escrow, incentives)
- Orchestrator services that marshal jobs and telemetry
- Sentinels & thermostats: safety, drift, and health
- Paymaster + policy layer for settlement routing

- CI v2 pipelines export audit artifacts and matrices
- Branch-protection guard blocks unsafe merges
- Companion workflows and status walls for visibility
- Reports feed consoles and owner workflows

Make the build system enforce governance and operational truth.

- CLI renders a live, machine-readable CI wall
- Produces Markdown/JSON artifacts for dashboards
- Mirrors GitHub branch protection requirements
- Local green wall → GitHub green wall
- Parity check between required contexts and CI outputs
- Refuses merges unless control invariants remain intact
- Codifies “deployable truth” into main
- Forces reproducibility and pinned toolchains
- API-level verification via GitHub REST endpoints
- Auditors corroborate results without local tooling
- Artefact digests trace back to commits and runs
- Reduces trust in any single operator or machine

Cryptographic attribution for binaries, tags, and upgrades.

- Allowlisted signers for releases and deployments
 - Keys rotated under policy; roles separated
 - Identities bound to ENS/DID registries
 - Multi-party approval for privileged actions
- Signed tags and versioned releases only
 - Deterministic build inputs and pinned dependencies
 - Artefact hashes recorded for traceability
 - Upgrade scripts consume the same provenance data
- Who signed what, when, and against which commit
 - Reproducible SBOMs for each release
 - Governance review packs (diffs + matrices)
 - Forensics-ready provenance for disputes

Parameterizable incentives that stay tethered to verified work.

- Fee matrix and splits (workers/validators/treasury)
 - α -WU tier multipliers and SLO scoring policy
 - Emission schedules and burn fractions
 - Paymaster policies and limits
- Jobs escrowed, then released on validation
 - Treasury routing and burn executed by policy
 - Operator withdrawals and validator rewards
 - Transparency dashboards for every flow
- Staking gates roles; slashing deters fraud
 - Rewards proportional to validated α -WU
 - Thermostats dampen runaway incentives and drift
 - Governance changes are auditable and reversible

Continuous verification across contracts, services, and releases.

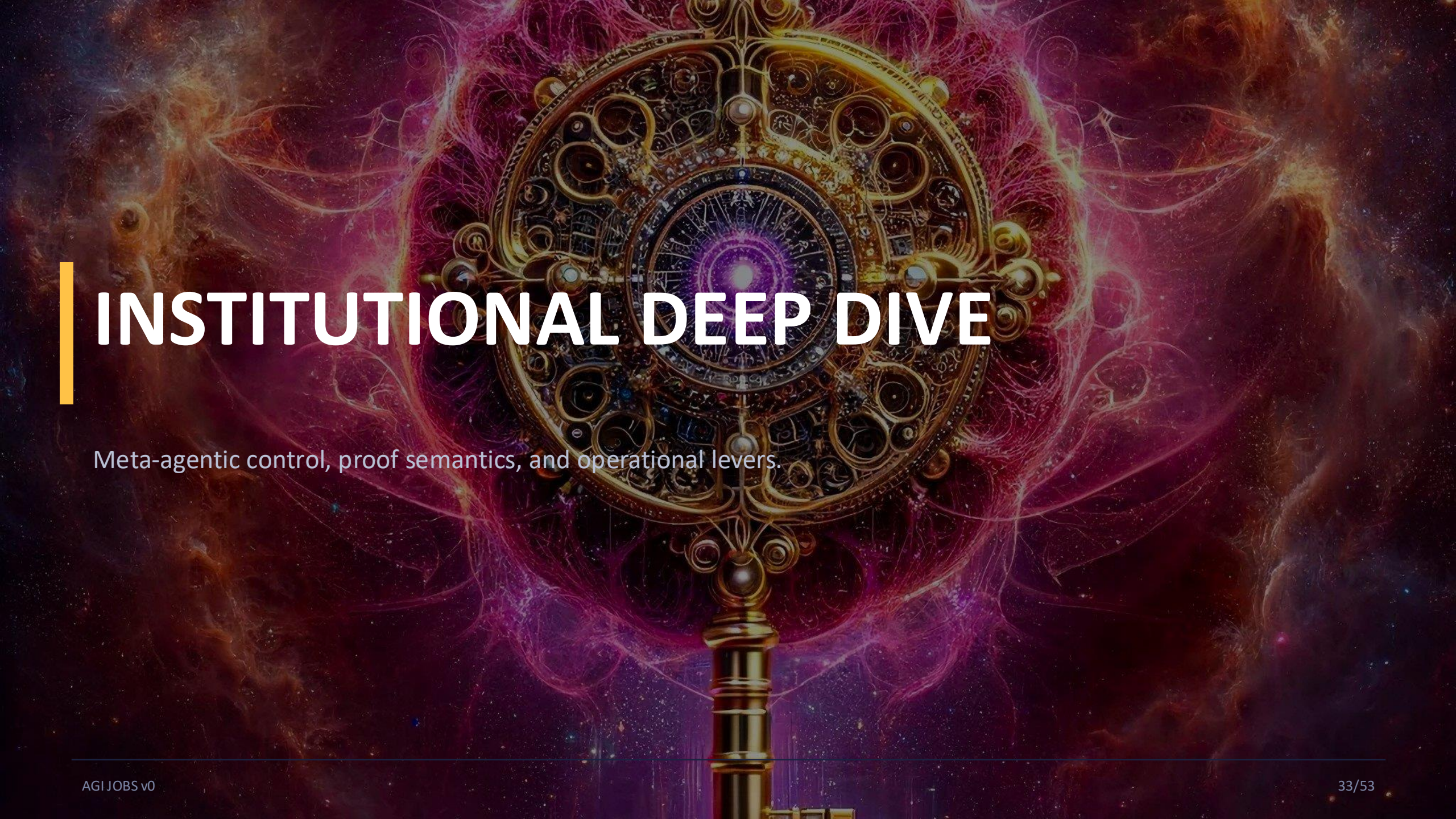
- Modular kernel with clear invariants and ownership
 - Static analysis, linting, and dependency pinning
 - SBOMs and reproducible builds by default
 - Privilege boundaries enforced through governance
- Unit + integration tests across job lifecycle
 - Fuzzing and adversarial scenarios for critical modules
 - CI gates for coverage and invariant checks
 - Simulation harnesses for economic stress
- Incident playbooks: pause, rollback, quarantine
 - Observability wall: metrics, logs, and alerts
 - Key custody, rotation, and access control
 - Upgrade rehearsals before deployment

Deployment runtime that executes jobs and emits settlement-grade proofs.

- Containerized runtime + metering sidecars
- Deterministic execution and artifact packaging
- Usage signing and submission to AGI Jobs kernel
- Role activation via stake and identity
- ENS-anchored node identity and role separation
- Owner-controlled upgrades and circuit breakers
- Runtime governance via peer coordination
- Autonomy bounded by policy and keys
- Dashboards for α -WU, SLO, and validator activity
- Health signals for sentinels and thermostats
- Tamper-evident logs for audits
- Operational truth exported as artifacts

Meta-agentic orchestration and evaluation built to produce verifiable work.

- Planners request jobs via the AGI Jobs control plane
 - Breaks objectives into verifiable tasks and artifacts
 - Emits structured evidence for validation
 - Interfaces with node runtime for deterministic execution
- Quality scoring and regression checks across epochs
 - Policy-aware routing by model tier and SLO
 - Feedback loops to improve job execution strategies
 - Simulations to pre-test upgrades and parameters
- Owners keep final authority: pause, upgrade, parameterize
 - Operators enforce policy boundaries and key custody
 - Auditors review artifacts and provenance packs
 - Autonomy remains bounded by governance



INSTITUTIONAL DEEP DIVE

Meta-agentic control, proof semantics, and operational levers.

Meta-Agentic Command Lattice

INSTITUTIONAL DEEP DIVE

A second-order system that governs autonomous work without sacrificing control.

Sense

- Observe environments, SLOs, and policy posture
- Forecast demand, risk, and capacity (α -WU)
- Detect drift, anomaly, and incentive stress

Decide

- Compile intent $\rightarrow$ JobSpecs (constraints + tests)
- Route work by tier, policy, and reputation
- Gate autonomy via budgets, allowlists, and keys

Settle

- Deterministic execution $\rightarrow$ proof bundle
- Peer validation $\rightarrow$ α -WU finalized
- Escrow release + Chronicle write (memory)

Stack: Agent ↔ Jobs OS ↔ Node

SYSTEM

One coherent surface from cognition to execution to settlement.

AGI-Alpha-Agent-v0

- Meta-agentic orchestration and evaluation loops
- Decompose objectives into verifiable JobSpecs
- Improve strategies using validated Chronicle
- Emits accountable traces and artifacts

AGIJobsv0

- Contracts kernel + paymasters + governance levers
- Job registry, escrow, staking/slashing, indices
- APIs, dashboards, simulations, CI v2 assurance wall
- Policy-bounded autonomy; owner retains override

AGI-Alpha-Node-v0

- Deterministic sandbox execution + metering sidecars
- Artifact packaging + signed telemetry/receipts
- Validator + sentinel services; observability wall
- Operator UX: deploy → register → participate

A job is not a prompt — it is a contract for verifiable outcomes.

Objective

- Clear goal, deliverables, and expected artifacts
- Budget in α -WU / \$AGIALPHA, with escrow bounds
- Policy tags: domain, tier, residency, tools

Constraints

- Environment pin (image digest, deps, seeds)
- Resource caps (GPU_s, memory, network)
- Tool allow/deny lists; external call boundaries

Acceptance

- Machine-checkable tests + success metrics
- SLOs: latency, uptime, deadlines
- Failure semantics: invalid $\rightarrow \alpha$ -WU = 0

Proof Bundle: Settlement-Grade Evidence

PROOF KERNEL

The object that survives audit, replay, and dispute.

Identity

- ENS/DID-anchored node + agent signatures
- Role-scoped permissions recorded at execution time
- Attributable receipts: who did what, when

Environment

- Container/image digest + SBOM / manifest
- Deterministic seeds, pinned deps, input hashes
- Policy context + version commitments

Artifacts

- Content-addressed outputs (hash/CID/URI)
- Logs, traces, metrics; metering report (α -WU inputs)
- Validator attestations + dispute window hooks

Validation: Commit–Reveal Attestation

PROOF KERNEL

Peer verification with bonded incentives; replay beats rhetoric.

Commit

- Validators lock hashed verdicts + scores
- Prevents herding; reduces bribery leverage
- Requires stake; slashing risk activated

Reveal

- Publish evidence: replay traces, test results, QV
- Check SLO_pass + policy compliance + metering plausibility
- Disagreements open disputes with evidence packs

Finalize

- Quorum closes; α -WU computed deterministically
- Escrow release + receipt issuance
- Reputation update + Chronicle write

Disputes & Slashing: Forensic Resolution

PROOF KERNEL

When reality disagrees, evidence wins.

Triggers

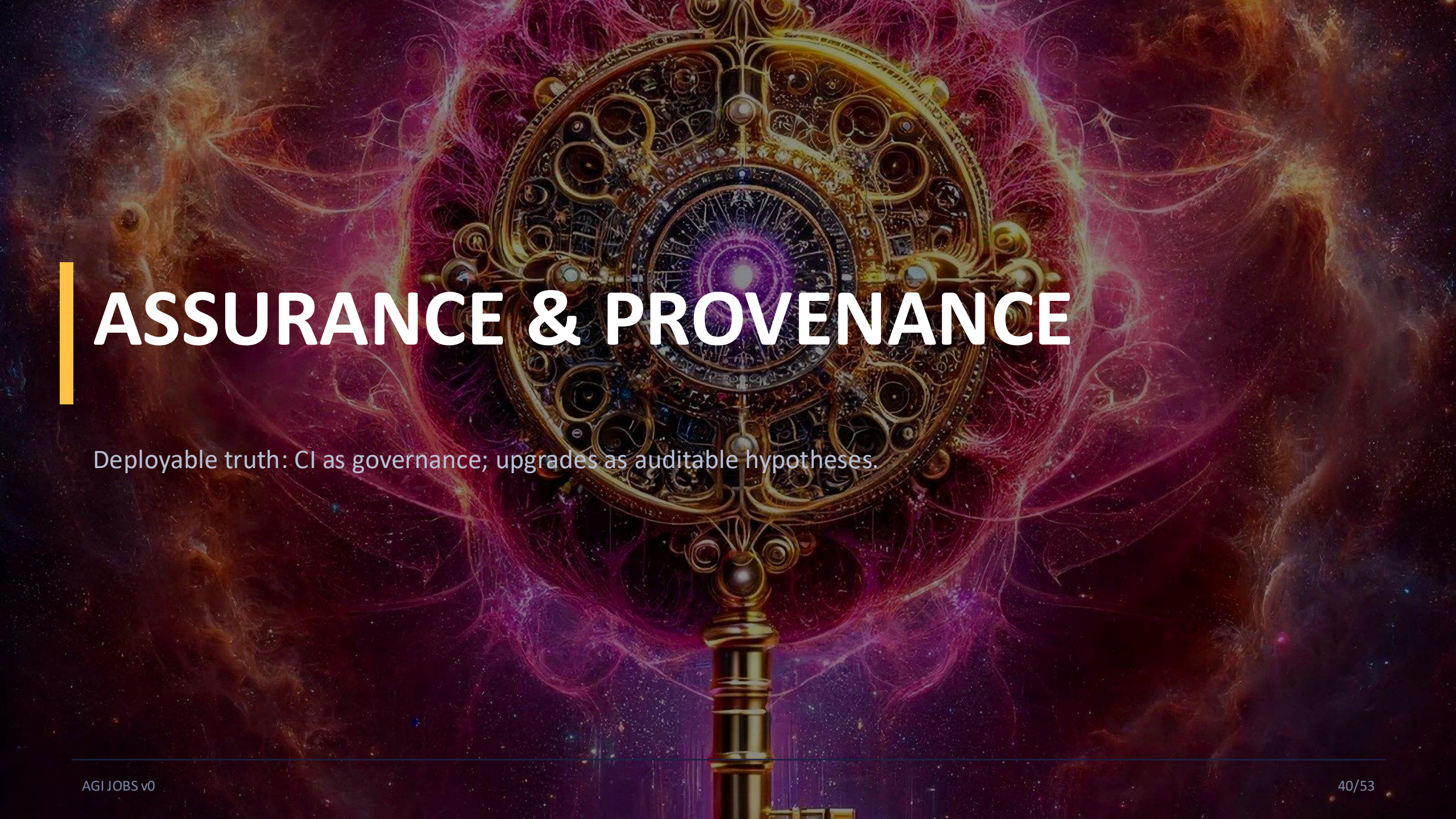
- Hash mismatch in deterministic replay
- Failed acceptance tests / policy violations
- Metering anomalies; inconsistent attestations

Evidence

- Expected vs observed outputs (content-addressed)
- Repro traces, logs, environment pins, timestamps
- Attestation transcripts (commit–reveal)

Outcome

- Fraud/invalid work → payout blocked ($\alpha\text{-WU} \rightarrow 0$)
- Mis-votes → validator slashed; stake redistributed/burned
- Chronicle records final state for future routing



ASSURANCE & PROVENANCE

Deployable truth: CI as governance; upgrades as auditable hypotheses.

Main remains deployable truth via required contexts and machine-readable status.

Wall

- Required checks: tests, lint, security, determinism
- Merge blocked unless wall is green
- Single operational surface for integrity

Parity

- Branch-protection drift detection vs canonical manifest
- GitHub settings corroborated via API checks
- Reduces trust in any single operator machine

Artifacts

- Reports exported (Markdown/JSON) for dashboards
- Provenance receipts link builds → commits → signers
- Audit packs generated continuously

Branch Protection Parity

ASSURANCE

Rule-of-law symmetry: config $\equiv$ enforcement.

Canonical

- Policy expressed as versioned manifests
- Required contexts and thresholds are explicit
- Changes reviewed like code

Enforced

- GitHub rules and CI outputs must match
- PR merges fail closed on mismatch
- Upgrades blocked without parity

Observable

- Status wall exposes checks, drift, and releases
- Auditors can verify without local tooling
- Evidence outlives teams and vendors

Who signed what, when, and against which commit — cryptographically attributable.

Signers

- Allowlisted release captains / guardians
- Role separation + key rotation under policy
- ENS/DID binds identities to signatures

Rules

- Signed tags + versioned releases only
- Pinned toolchains + SBOMs for reproducibility
- Upgrade scripts consume provenance data

Receipts

- Artifact hashes recorded; diffs + matrices exported
- Multisig-ready bundles; air-gapped workflows
- Forensics-ready provenance for disputes

Policy Versioning & Upgrade Gates

ASSURANCE

Autonomy evolves only through traceable, reversible change.

Policy

- Versioned parameters: fees, tiers, SLO scoring
- Routing constraints: allowlists, budgets, roles
- Policies are part of proof bundles

Timelock

- Upgrade queue with delay for review
- Pre-deployment simulations + stress tests
- Emergency brakes override schedule

Rollback

- Deterministic revert paths; quarantine modes
- Evidence bundles enable time-travel audits
- Integrity maintained across upgrades



ECONOMIC CONTROL SURFACE

Work metered, priced, and settled with feedback control — aligned to verified output.

Thermostat: Hamiltonian Feedback

ECONOMICS

Stability mechanism to dampen incentive runaway while throughput scales.

Signals

- Validated α -WU / epoch (throughput)
- Dispute rate, validator variance, SLO drift
- Treasury flows, burn/emission ratios

Actuators

- Fee splits, burn fractions, emission schedules
- Tier multipliers and scoring weights (QV/SLO)
- Quorum and slashing parameters

Guardrails

- Fail closed under high uncertainty
- Budget caps and rate limits by policy
- Governance logs every parameter change

Fee Matrix & Incentive Splits

ECONOMICS

Make honesty the dominant strategy; route value to verified contribution.

Requester

- Escrow funds jobs; defines acceptance and SLOs
- Pays only for validated proof (no counter-party risk)
- Receives artifacts + settlement receipts

Worker

- Executes deterministically under policy
- Earns payout proportional to validated α -WU
- Reputation compounds with proofs

Validator

- Attests via replay + scoring; stake-bonded
- Earns fees for catching errors and confirming truth
- Misbehavior becomes negative-sum (slashing)

Treasury & Backstops

ECONOMICS

A long-horizon reserve that funds robustness, audits, and ecosystem growth.

Reserve

- Transparent inflows from fees/splits (policy-tunable)
- Auditable allocations and sweeps
- Backstop for rare systemic shocks

Safety Budget

- Security audits and incident response readiness
- Validator/sentinel incentives during stress
- Monitoring and evaluation infrastructure

Ecosystem

- Grants for tools, dashboards, integrations
- Templates for regulated domains
- Bootstraps a market for provable work



ENTERPRISE ADOPTION

Start small, prove determinism, then scale autonomy only as verification stays ahead.

Adoption Playbook

ADOPTION

A controlled ramp from pilot to production to market-scale work.

Phase 1

- Proof-first sandbox (non-critical workloads)
- Deterministic runs + evidence bundles
- Read-only dashboards; export audit packs

Phase 2

- Independent validators; dispute windows
- Escrow settlement for internal services
- Policy templates: budgets, allowlists, scopes

Phase 3+

- Tiered markets + reputation routing
- Hybrid/private deployments with selective verification
- Expand autonomy only after safety gates pass

Controls Mapping

ADOPTION

Identity, evidence, and change management align with institutional oversight.

Identity & Access

- Role-scoped permissions (least privilege)
- ENS/DID binding for non-repudiation
- Key rotation and separation of duties

Evidence & Audit

- Replayable proof bundles as audit objects
- Tamper-evident logs + content addressing
- Exportable receipts for SIEM/GRC pipelines

Change Control

- CI wall enforces governance before merge
- Signed releases + timelocked upgrades
- Immutable Chronicle for reconstruction

Operational Drill: Fail-Closed Control

ADOPTION

When uncertainty spikes: pause → isolate → recover — proven under stress.

Pause

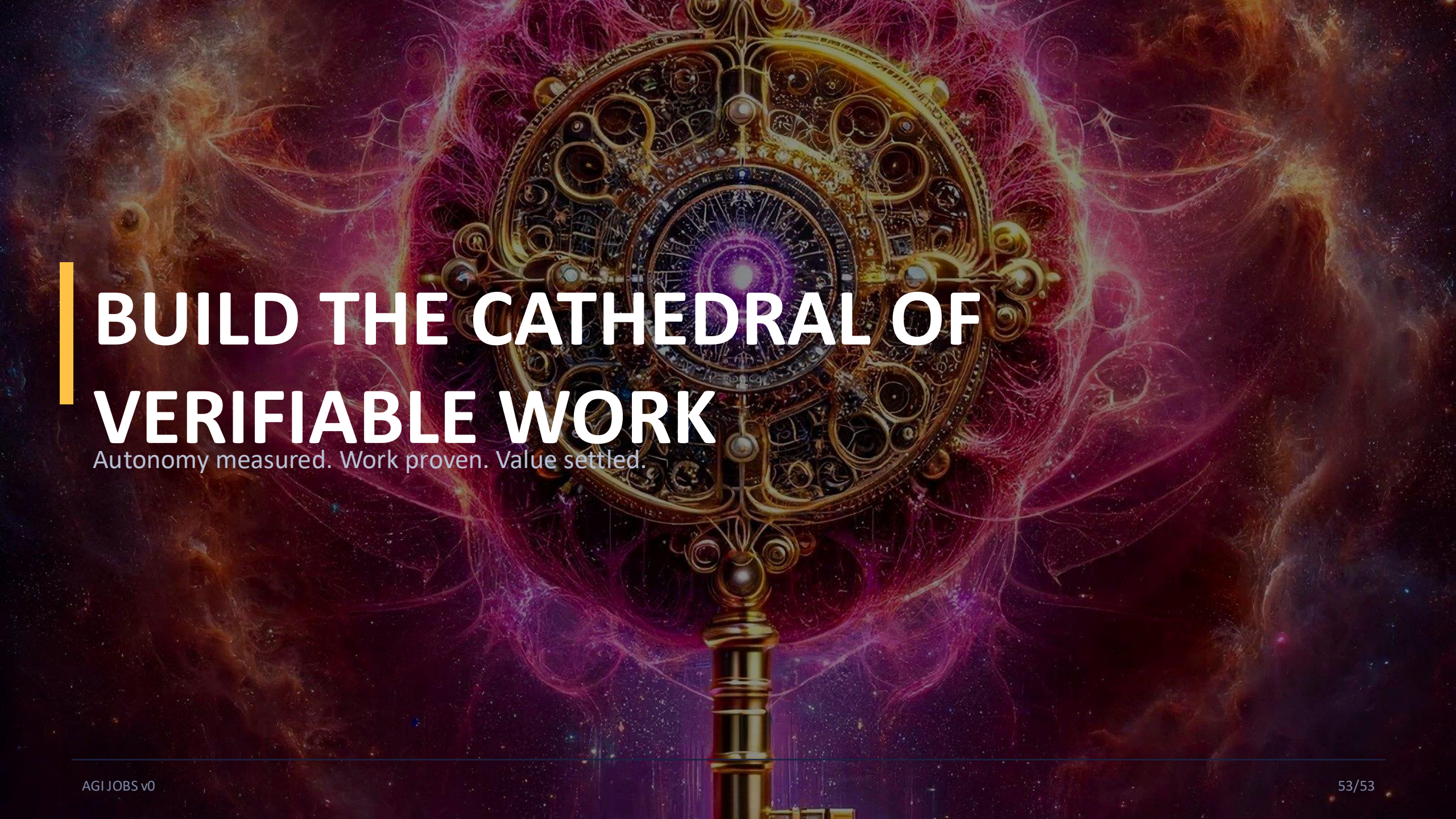
- Emergency stop via SystemPause
- Freeze payouts; halt risky actions
- Preserve evidence; notify governance

Quarantine

- Isolate nodes/validators; tighten policy gates
- Rotate keys; lock allowlists; reduce scope
- Increase validation thresholds temporarily

Recover

- Deterministic replay + root-cause analysis
- Roll back upgrades; re-issue safe policies
- Resume with measured gates and dashboards



BUILD THE CATHEDRAL OF VERIFIABLE WORK

Autonomy measured. Work proven. Value settled.